STUDY NOTES

Agentic Architecture

Every module on one page each, with diagrams. Each page ends with a line to say out loud in an interview.

1. GenAI, Agents & the Maturity Model
2. Agent-Ready LLMs
3. The Adaptation Spectrum: RAG → Fine-tuning → Grounding
4. Agent Architecture & Interaction Models
5. Coordination I: Delegation & Topologies
6. Coordination II: Collective Intelligence
7. Explainability & Compliance Patterns
8. Robustness & Fault Tolerance Patterns
9. Human-Agent Interaction Patterns
10. Agent-Level Patterns
11. System-Level Patterns for Production
12. Building Agents That Learn
13. The Roadmap & Frameworks

GenAI, Agents & the Maturity Model

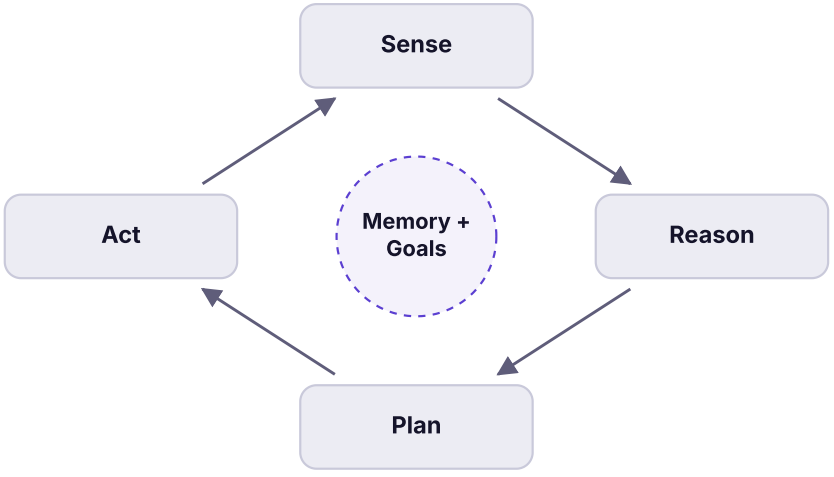
1. Three things, not one

- **LLM call**: a single request and response. Powerful but stateless and passive: exactly one turn
- **Automated workflow**: a predefined sequence where **your code decides the path** and LLM calls fill specific slots (classify, extract, draft). Predictable and auditable
- **AI agent**: an LLM placed in a loop with tools, where **the model decides** what to do next to reach a goal
- If the path is knowable in advance, a workflow is cheaper, faster and easier to debug. Reserve agents for genuinely open-ended tasks

2. Agent anatomy

- **Goals**: what it's trying to achieve
- **Sense**: perception, gathering context from APIs, databases, sensors ('context is king')
- **Reason**: the LLM cognition core · **Plan**: decide the next steps · **Act**: execute through tools
- **Memory**: state and past experience · **Coordinate**: only in multi-agent systems

✎ DIAGRAM



The agent loop: the result of each action becomes new context, grounded in memory and goals at every step.

3. The GenAI Maturity Model (L0 to L6)

It's not just a description of where you are; it's a **build order**, because each level is reached by implementing specific patterns.

✎ DIAGRAM



Level 4 is where outputs become trustworthy; agents (L5 to L6) need AgentOps on top.

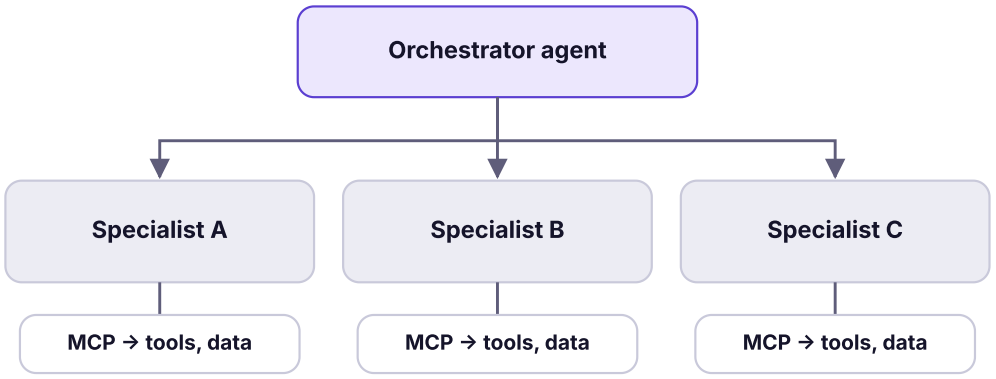
Each level is unlocked by specific patterns.

- **Pattern-first architecture**: pick a target capability ('fully auditable and safe for financial transactions'), reverse-engineer the four or five patterns that deliver it, and build exactly those

4. The agentic stack

- **Function calling** (the agent acts): you give the model tools with names, descriptions and parameter schemas; it emits a structured JSON call; your code runs it and returns the result. The description is what it routes on
- **MCP** (Model Context Protocol, agent → tools, vertical): a standard for describing, discovering and invoking tools and data sources
- **A2A** (Agent-to-Agent, agent → agents, horizontal): a shared language for delegating tasks between agents across frameworks and vendors. Discovery via an **agent card** (advertises capabilities) on an **agent server** (the endpoint)

✎ DIAGRAM



A2A connects the orchestrator to specialists (horizontal); MCP connects each agent to its tools (vertical).

5. Why production is hard

- Low-latency inter-agent communication, diverse data, reliable tool use, hallucination, **observability** into why a multi-agent system did something, an expanded attack surface, and the **AgentOps** discipline to run it all
- You don't need every pattern, only the ones that close the gap between your demo and the reliability your use case demands

SAY IT

"MCP connects an agent to its tools; A2A connects agents to each other. I'd only build an agent where a workflow can't do the job."

Agent-Ready LLMs

1. Choosing a model for an agent

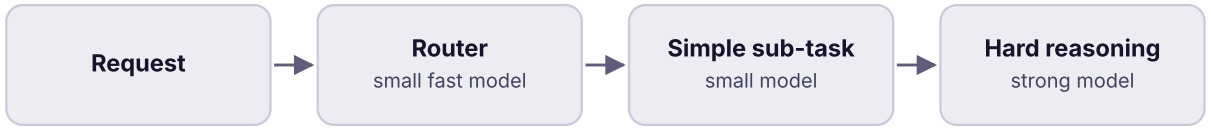
An agent's reasoning core has different needs from a chatbot's. The best benchmark score isn't the criterion; **reliable tool use** usually is.

- **Context window**: must hold system prompt, tool schemas, memory, retrieved context and the running trace
- **Size & specialisation**: a smaller, faster model that routes tools reliably can beat a larger one
- **Native tool calling**: decisive. A model that can't reliably emit valid, correct calls isn't agent-ready
- **Robustness**: resists injection, formats consistently, because it runs unattended in a loop
- **Adaptability**: can it be tuned (PEFT) if the role needs it?
- **Cost & latency**: multiplied by every call in every loop

2. Select per role

- Cheap, fast model → routing, extraction, simple sub-tasks
- Strong model → hard reasoning and planning
- This **tiering** is one of the biggest cost and latency levers you have

✎ DIAGRAM



Route each step to the cheapest model that does it reliably.

3. Serving options

- **Cloud APIs**: fastest start, no ops, elastic; less control over data residency
- **Self-hosted**: full control, data stays in your environment; you own GPUs, serving and upgrades
- **Edge**: small on-device models for privacy, offline use and ultra-low latency

4. Three optimisation axes

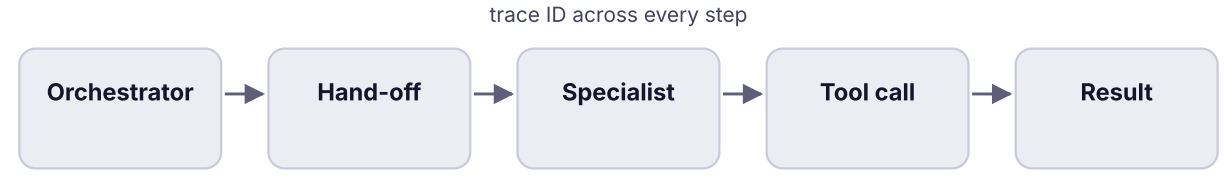
- **Latency**: streaming, smaller or distilled models for simple steps, caching stable prefixes, fewer tool round-trips
- **Throughput**: batching and concurrency
- **Cost**: route easy work to cheaper models, trim context, cap output and reflection loops

> ⚠ In agents, latency is often dominated by **tool round-trips**, not the model. Make tools fast, idempotent and coarse enough to avoid chatty back-and-forth.

5. AgentOps

- LLMOps extended for autonomy: log every step of the loop, trace tool calls and inter-agent messages, version prompts and tools, manage deploy / update / rollback, and watch cost and reliability
- Without **step-level observability**, a multi-agent failure is undebuggable: you need a trace across orchestrator and specialists to see which agent, tool or hand-off caused it

✎ DIAGRAM



Step-level tracing is what makes a multi-agent failure debuggable.

SAY IT

"I select a model per role: a cheap one for routing and simple steps, a strong one for hard reasoning, and I trace every step so failures are debuggable."

The Adaptation Spectrum: RAG → Fine-tuning → Grounding

1. Lightest option first

✎ DIAGRAM



Cost, effort and risk rise as you move right. Stop at the first rung that meets the need.

The adaptation spectrum.

- **In-context learning (ICL)**: instructions and examples in the prompt; no training run
- **RAG**: inject external knowledge at run time ('the model doesn't know our facts')
- **Grounding**: link outputs to sources for trust and auditability
- **PEFT** (LoRA, adapters): tune a small slice of weights for style, format or terminology. Cheap, fast, low-risk
- **Full fine-tuning**: retrain substantial weights for deep specialisation. Costly, easy to overdo
- Rule: **RAG for facts, fine-tuning for behaviour and style**

2. The Agentic AI Maturity spectrum

1 **Basic**: single agent, fixed workflow, predefined tools

2 **Dynamic tools**: chooses from a set of approved tools per problem

3 **Introspective**: **ReAct** (alternate reasoning and acting) + **Reflexion** (self-critique and correction)

4 **Multi-agent**: specialists coordinate in supervised, auditable workflows

5–6 **Collaborative**: decentralised negotiation, consensus, swarm

- Each stage brings new failure modes, so each calls for new patterns: dynamic tools need routing, introspection needs self-correction, multi-agent needs coordination and conflict resolution

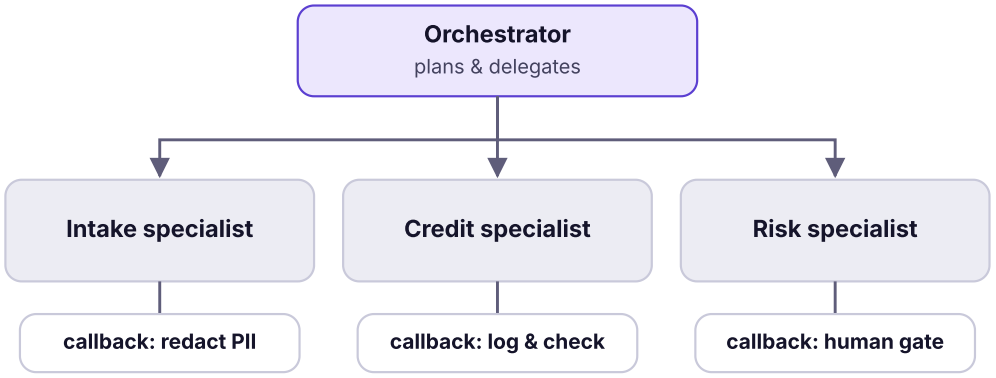
3. RAG as sensing

- Support agent → latest product and policy docs · analyst agent → current market data · compliance agent → current regulations
- For agents, retrieval is **perception**: how the agent sees the current state of its domain instead of relying on frozen, generic weights

4. Hierarchical architecture

- An **orchestrator** decomposes the goal and delegates; **specialists** each own one sub-task with their own tools; results flow back up
- **Callbacks** wrap every hand-off: hooks that inspect, redact, filter or block input and output. This is where governance lives: PII redaction, safety checks, human-in-the-loop gates, the audit trail

✎ DIAGRAM



Specialists do the work; callbacks on every hand-off make the hierarchy controllable.

SAY IT

"I'd reach for the lightest adaptation that works: prompting, then RAG, then grounding, then PEFT, and full fine-tuning last."

Agent Architecture & Interaction Models

1. The environment an agent perceives

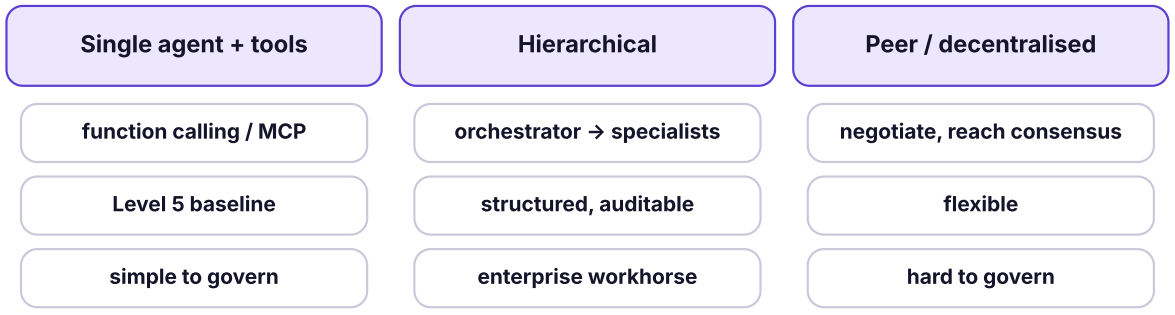
- **Digital context**: unstructured data (text, images), structured data (databases, **knowledge graphs**), vector stores for similarity search
- **Physical context**: sensors and actuators for embodied agents
- **Shared memory / communication hub**: in multi-agent systems, where agents exchange information, plans and goals

2. Memory is not one thing

- **Short-term (working) memory**: the current task and trace
- **Long-term memory**: past experience and learned facts
- **Shared memory**: team state
- **Knowledge graphs** give a structured understanding of entities and relationships that plain vector search can't, enabling deeper reasoning

3. Interaction models

✎ DIAGRAM



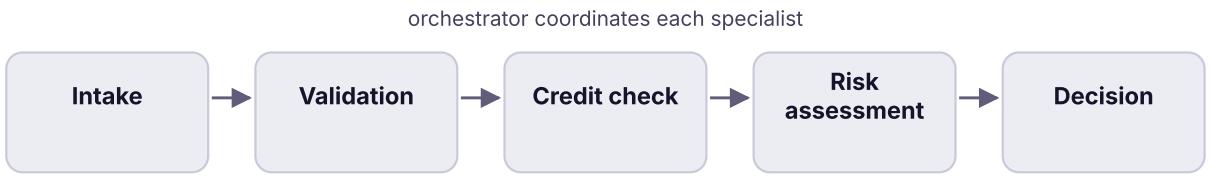
From simple to collaborative.

4. Two case studies

- **Travel planning agent**: one agent takes a goal ('a Tokyo trip within budget'), senses via flight and hotel APIs, plans an itinerary, books it. A clean single-agent example of the full loop
- **Agentic loan processing**: intake → validation → credit check → risk assessment → decision. As one agent it's a brittle monolith; as a multi-agent system an orchestrator coordinates specialists via A2A, which is more robust, maintainable and scalable

- Loan processing recurs because it has everything: multi-step flow, high stakes (governance, audit), clear specialisation, and exception paths

✎ DIAGRAM



The loan pipeline: each stage is a natural specialist agent.

SAY IT

"Hierarchical orchestration is the enterprise default because it's structured and auditable; I'd only go decentralised when the problem genuinely rewards it."

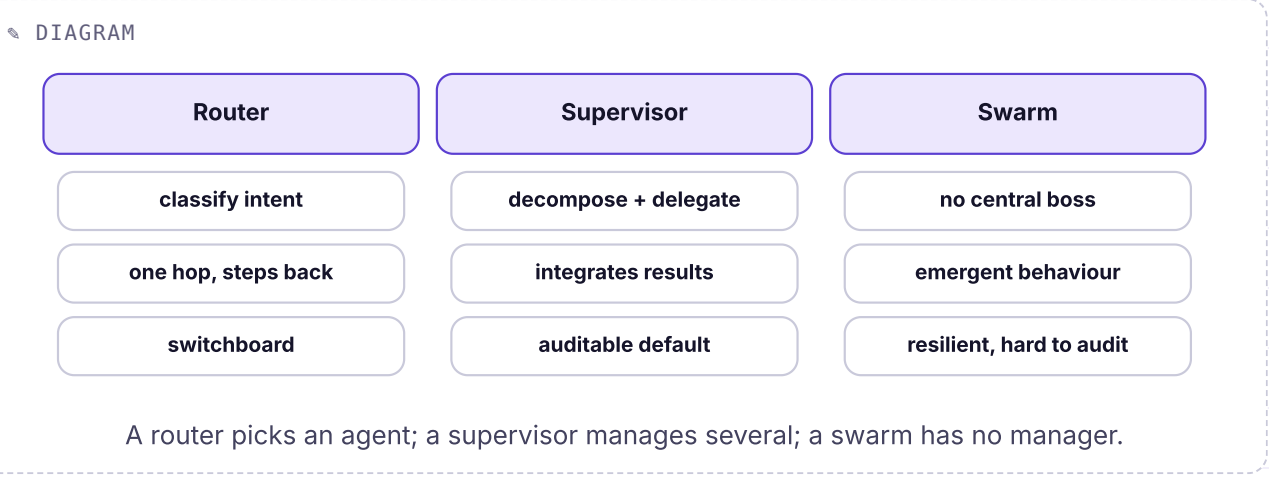
Coordination I: Delegation & Topologies

1. The principle

Every coordination pattern answers one question: **who does what, and who decides?** Use the **least coordination** that solves the problem, because every mechanism adds latency, cost and failure surface. Start structured; earn decentralisation.

2. Router, supervisor, swarm

- **Agent Router**: a lightweight classifier reads the request's intent and dispatches it to the right specialist, then steps back. Single point of failure; needs a fallback for ambiguous requests
- **Supervisor**: a central orchestrator breaks the goal into sub-tasks, delegates, and integrates the results. Predictable and auditable, but a bottleneck and single point of failure. **The enterprise default**
- **Swarm**: many agents follow simple local rules with no boss; global behaviour emerges. Resilient and adaptive, but hard to predict, govern and audit



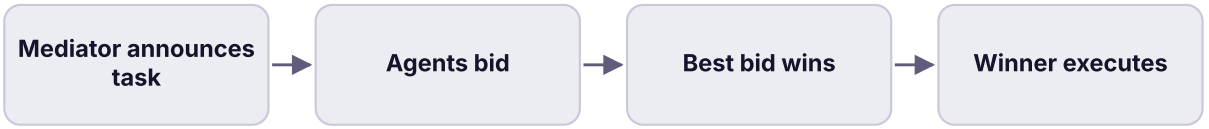
A router picks an agent; a supervisor manages several; a swarm has no manager.

3. Composition topologies

- **Blackboard Knowledge Hub**: agents read and write a shared workspace; a controller invokes whichever specialist can contribute next (e.g. collaborative medical diagnosis). Flexible; shared state can become a contention problem
- **Contract-Net Marketplace**: a mediator announces a task, agents **bid**, the best bid wins (e.g. choosing a cloud provider on cost and latency). Great for dynamic allocation; bidding adds latency
- **Supervision Tree with Guarded Capabilities**: supervisors monitor children and restart or contain failures; each node gets only the capabilities it needs (e.g. a

resilient web scraper). Strong fault isolation and least privilege

✎ DIAGRAM



Contract-Net: dynamic allocation by bidding.

SAY IT

"A router picks an agent and steps back; a supervisor stays in charge and integrates the results. Supervisor is the enterprise default because auditability matters more than emergent flexibility."

Coordination II: Collective Intelligence

1. Sharing the work, the knowledge and the tools

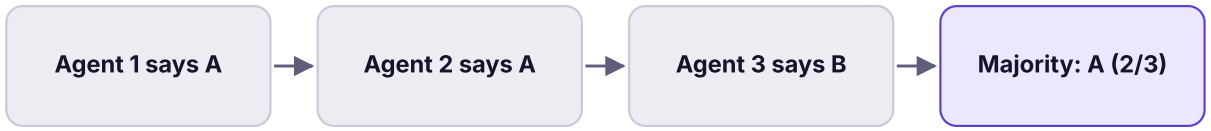
- **Multi-Agent Planning**: agents (or a planner) decompose a goal, assign sub-tasks and sequence dependencies before executing. Watch for re-planning when reality diverges
- **Knowledge Sharing**: a shared store of solved problems agents contribute to and draw from, so the team stops re-solving the same things
- **Tool Routing**: a layer that sends each tool request to the right provider or agent when there are many of both
- Common thread: these externalise what one agent keeps in its head (the plan, the knowledge, the tool map) into shared structures

2. Consensus and negotiation

- **Consensus**: cooperative agents debate, critique and vote (majority, weighted, structured debate) to converge on one answer, e.g. analysts converging on a forecast
- **Negotiation**: self-interested agents exchange offers and counter-offers until they reach a deal over a shared resource. Needs protocols, deadlines and fallback rules

> ⚠ Voting only reduces error when agents' mistakes are **independent**. Three copies of the same model with the same blind spot just repeat the mistake.

✎ DIAGRAM



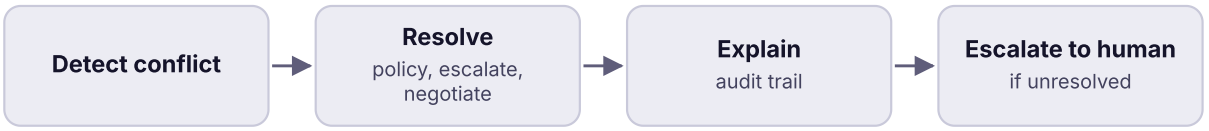
Consensus helps only if the agents' errors are independent.

3. Keeping order under contention

- **Resource Allocation**: market-based, priority-based or optimisation-driven assignment of limited resources (e.g. robots across factory tasks)
- **Conflict Resolution**: when agents reach incompatible decisions: escalate to a supervisor, apply a policy, negotiate, or use game theory. Essentials: **detect** the conflict, **explain** the resolution (audit trail), and keep a **human escalation path**

- **Formation Control**: local rules that keep physical agents in a coordinated shape (e.g. a drone swarm covering a field)

✎ DIAGRAM



Detect, explain, escalate: the governance instinct that recurs across the book.

SAY IT

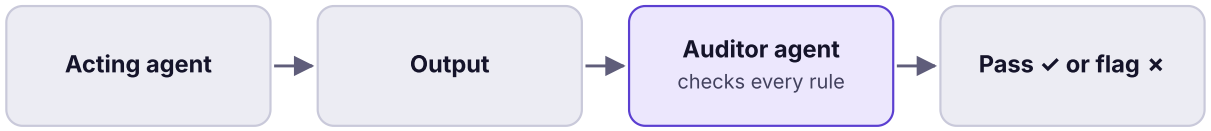
"Consensus is cooperative agents converging on the best answer; negotiation is competing agents converging on a deal. Either way I'd log the resolution and keep a human escalation path."

Explainability & Compliance Patterns

1. Staying on the rules

- **Instruction Fidelity Auditing**: a separate auditor checks the output (and reasoning) against every stated instruction and flags any skipped or violated. Example: confirming an e-commerce agent applied exactly the eligible discounts
- Why a separate auditor? An agent **rationalises its own mistakes**; objective checking needs an external evaluator
- **Persistent Instruction Anchoring**: re-inject the non-negotiable constraints at every step of a long run so they stay salient (fights lost-in-the-middle). Anchor the critical constraints only, because it costs context

✎ DIAGRAM



The agent that acts shouldn't be the one that audits.

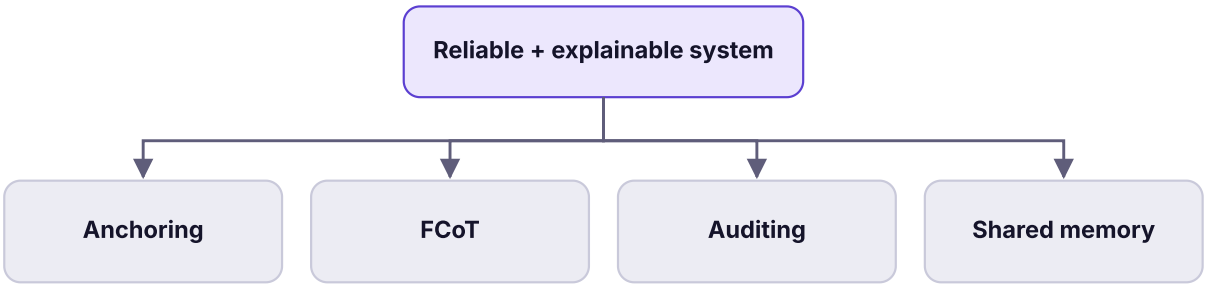
2. Making reasoning inspectable

- **Fractal Chain-of-Thought (FCoT)**: structured reasoning embedded at every level (orchestrator and each specialist), composing into one auditable chain, so a reviewer can trace why the system concluded something at any depth
- **Shared Epistemic Memory**: a common, versioned knowledge store all agents read and write, so they act from one consistent picture of the world (e.g. a supply-chain disruption response)

3. Compose for systemic reliability

- Anchor the constraints + embed FCoT for transparency + audit for fidelity + share epistemic memory for consistency → a system that is both reliable and explainable (maturity Level 4)
- In the loan use case, FCoT guides the agent's reasoning so every approve or reject decision comes with legible, auditable reasoning

✎ DIAGRAM



Explainability is designed into how the agent reasons, not added as a report later.

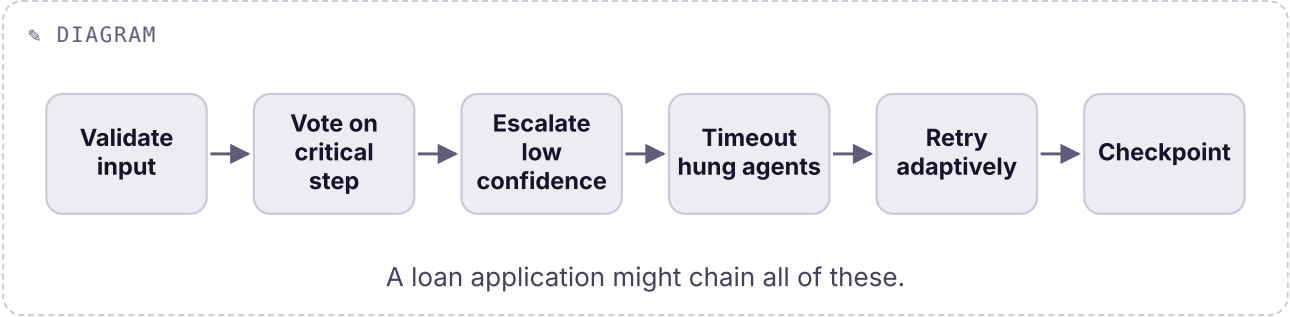
SAY IT

"Explainability isn't a report you add at the end; it's designed into how the agents reason, with a separate auditor checking the output against every instruction."

Robustness & Fault Tolerance Patterns

1. Robustness is a spectrum

No single pattern makes a system robust. You **chain** fault-tolerance patterns to reach the reliability the use case needs, and measure it: success rate under fault injection, mean time to recovery, graceful-degradation coverage.



A loan application might chain all of these.

2. Reliability through redundancy

- **Parallel Execution Consensus**: run the critical task on several agents and require agreement before proceeding
- **Majority Voting Across Agents**: take the majority answer of N agents

> ⚠ Voting multiplies cost by N and only helps when failures are independent. Use it on critical, hard-to-reverse decisions (a loan approval), not every step.

3. Keeping a run alive

- **Watchdog Timeout Supervisor**: kills or restarts any agent past its deadline, so one stuck agent can't freeze the system
- **Delayed Escalation Strategy**: on low confidence, try lighter recovery first, then a stronger model or a human
- **Adaptive Retry with Prompt Mutation**: analyse why it failed and change the approach before retrying, instead of repeating the same failing call
- **Auto-Healing Agent Resuscitation**: detect a crashed agent and restart it with its state
- **Incremental Checkpointing**: save state after each stage so a long pipeline resumes from the last good point

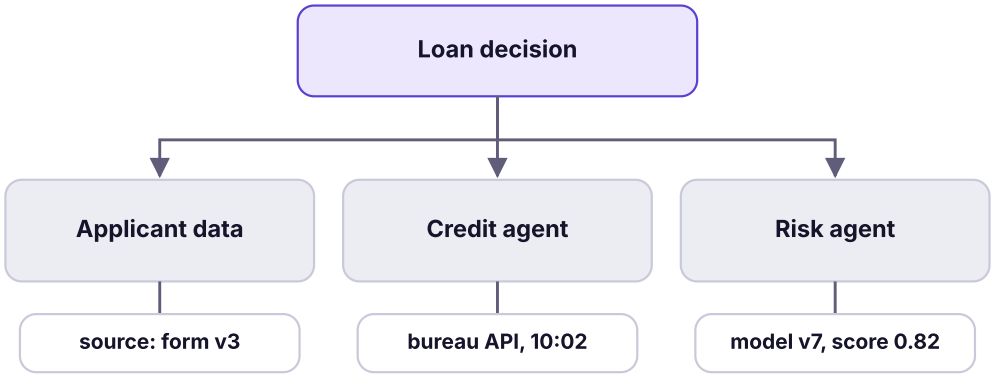
4. Security patterns

- **Agent Self-Defense**: the agent spots and neutralises adversarial input aimed at it (e.g. injection in feedback it summarises)
- **Agent Mesh Defense**: defence in depth; a compromised agent is contained by its peers and can't reach sensitive tools or data
- **Execution Envelope Isolation**: run risky actions such as code execution in a sandbox

5. Graceful degradation and auditability

- **Rate-Limited Invocation** (respect downstream limits) · **Fallback Model Invocation** (fail over to a secondary model) · **Trust Decay and Scoring** (down-weight flaky agents over time) · **Canary Agent Testing** (upgrade a small slice first) · **Optimising Translation Overhead** (summarise once, don't re-pass large documents)
- **Causal Dependency Graph**: records which inputs, agents and steps caused each output, so any decision can be traced afterwards. The observability backbone AgentOps needs

✎ DIAGRAM



A causal dependency graph lets you answer 'why was this loan rejected?' after the fact.

SAY IT

"Robustness comes from chaining patterns: validate inputs, vote on the critical decision, escalate low confidence, time out hung agents, retry adaptively, and checkpoint progress."

Human-Agent Interaction Patterns

1. Autonomy is a dial, not a switch

Five patterns describe every hand-off between people and agents, each with a different trust and control profile.

✎ DIAGRAM

Pattern	Who initiates → who acts	Example
Agent Calls Human	agent escalates to a person	loan agent pauses on an ambiguous case
Human Delegates to Agent	person hands over a goal	delegating market research
Human Calls Agent	person invokes for a task	checking an order status
Agent Delegates to Agent	agent hands a sub-task on	lead agent splits a financial analysis
Agent Calls Proxy Agent	agent talks to another org's proxy	cross-company loyalty programme

The five human-agent interaction patterns.

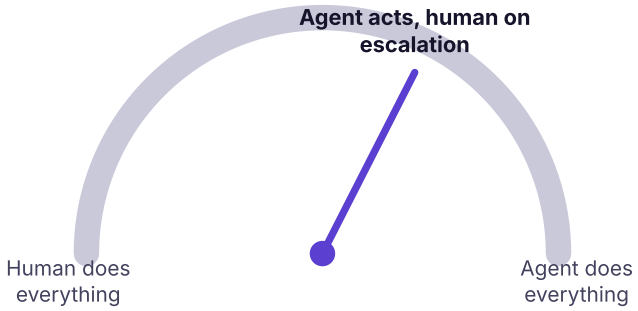
2. Agent Calls Human: the safety keystone

- When confidence is low or stakes are high, the agent **pauses and escalates** instead of acting
- You get most of the automation benefit while a human backstops the consequential decisions
- Design the **escalation triggers** deliberately: confidence thresholds, risk categories, ambiguity
- It's also part of the business case: production systems automate 60 to 80% and escalate the rest

3. Agent Calls Proxy Agent

- Makes cross-organisation collaboration possible: your agent never touches the other company's internals, only a proxy that represents them, preserving boundaries and trust

✎ DIAGRAM



Set the dial by the cost of an error and whether it can be reversed.

SAY IT

"Autonomy is a dial: the agent acts on routine cases and escalates to a human when confidence is low or the stakes are high, with the triggers designed deliberately."

Agent-Level Patterns

1. Master the single agent first

Before adding multi-agent complexity, build one agent that's genuinely production-worthy. Five patterns stack into it:

1 **Single Agent Baseline**: one LLM plus tools on a well-defined task (e.g. a simple loan-approval agent)

2 **Agent-Specific Context and Memory**: state so it stays coherent across a session (remembers the applicant)

3 **Sensing with RAG**: retrieval as perception, grounded in current private facts (checks live policy)

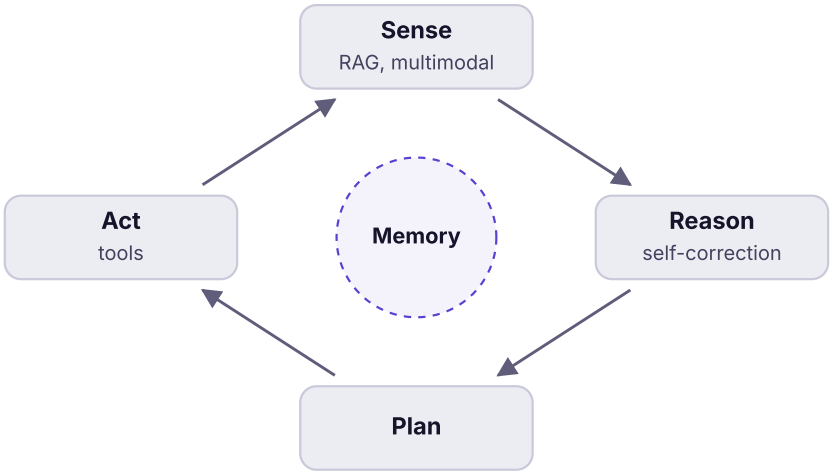
4 **Structured Reasoning and Self-Correction**: plans, critiques its own draft against criteria, revises before acting (ReAct / Reflexion as an internal pattern)

5 **Multimodal Sensory Input**: senses beyond text (reads a scanned application form)

2. They map onto the anatomy

- Memory → **Memory** · RAG and multimodal input → **Sense** · reasoning and self-correction → **Reason / Plan** with an internal feedback loop
- Each maps to a measurable success metric

✎ DIAGRAM



The agent-level patterns are the anatomy made concrete and reliable.

SAY IT

> *"I'd master one reliable agent before adding more: baseline, memory, sensing with RAG, self-correction, then multimodal input if the job needs it."*

System-Level Patterns for Production

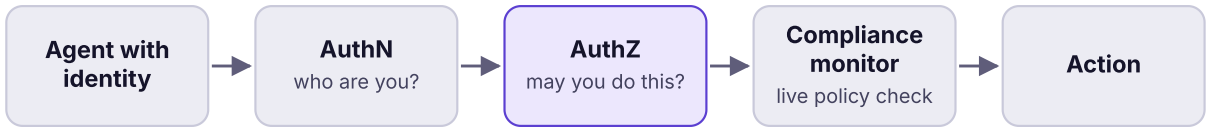
1. The plumbing around the agents

- **Tool and Agent Registry**: a discoverable catalogue of tools and agents (capabilities, schemas), so the system binds to the right one at run time instead of hard-coding (e.g. procurement discovering supplier agents). The enterprise version of the A2A agent card
- **Real-Time Compliance Monitoring**: every action checked against policy as it happens, blocked or flagged live (e.g. a healthcare agent screened for patient-data leakage)
- **Agent Authentication and Authorization**: agents have identities and scoped permissions; each request is authenticated and authorised before it runs (least privilege)
- **Event-Driven Reactivity**: agents wake on events (messages, webhooks, state changes), not only on request (e.g. a support agent woken when a ticket arrives)

2. Identity is the foundation of agent security

Once agents act autonomously and call tools, 'which agent is allowed to do what?' becomes a first-class question. Authentication + authorisation + real-time compliance monitoring let autonomous agents operate without exceeding their authority.

✎ DIAGRAM



Every agent action passes identity, permission and live policy checks.

SAY IT

"Agent authentication, scoped permissions and real-time compliance monitoring are what let agents act autonomously without exceeding their authority."

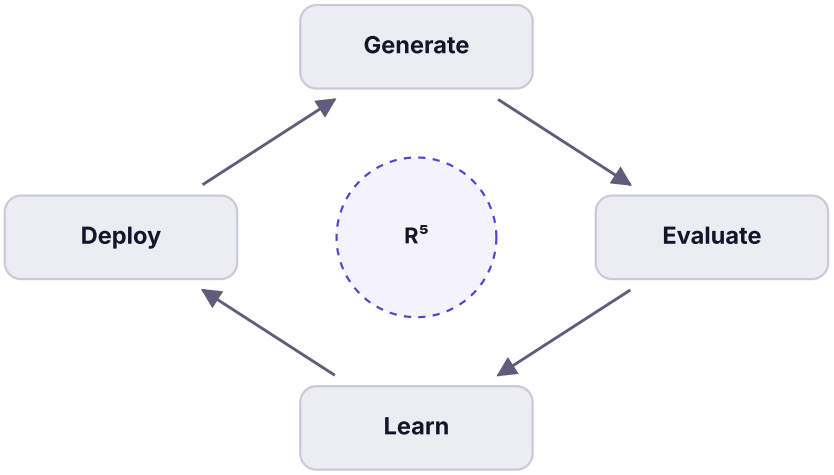
Building Agents That Learn

1. The Self-Improvement Flywheel

- **Generate** a solution → **Evaluate** it against quality criteria → **Learn** (update or tune) → **Deploy** the improvement → repeat
- **Evaluate** is the crucial stage: a system can only improve what it can measure
- **Learn** must balance **exploitation** (refining known-good strategies) and **exploration** (trying new ones), or it stagnates

> ⚠ Learning from bad data or a hallucinating evaluator turns the flywheel into a vicious cycle of degradation.

✎ DIAGRAM



The flywheel is the what; R^5 is the how that keeps it safe.

2. The R^5 model

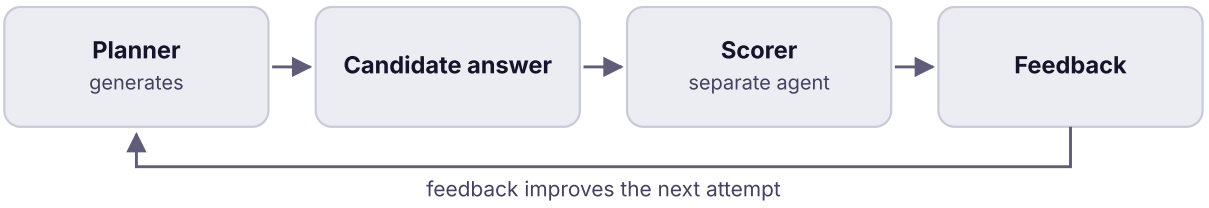
- **Relax**: actively manage context and latency so agents stay coherent under load
- **Reflect**: deliberate self-critique checkpoints, improving mid-run without retraining
- **Reference**: provenance (citations, retrieval traces) so outputs are attributable
- **Retry**: adaptive, reasoned error handling, not blind repetition
- **Report**: quantify factuality, consistency and process quality to close the loop

3. Patterns for learning safely

- **Planner + Scorer (hybrid) architecture**: separate the generator from the evaluator, because an agent judging itself rationalises its errors. The foundational pattern

- **Custom evaluation metrics**: task-specific scores, e.g. the quality of each step, not just the final answer
- **Preference-controlled synthetic data**: training data steered towards desired preferences
- **Tuning**: SFT and PEFT for capability, then **DPO** (learn from better-vs-worse pairs) for alignment
- **Coevolved training**: generator vs critic, both improving · **Red teaming**: attack your own agents first

✎ DIAGRAM



Never let one agent both create and grade its own work.

4. Tokenomics and ROI

- Multi-agent systems fan out into many calls that each resend context, so **cost scales with agents × steps**, not just tasks. Levers: route easy sub-tasks to cheaper models, trim and cache context, cap output and reflection loops, batch async work, track cost per task
- **ROI** = tasks × time saved × loaded rate × **realistic automation rate** − run and ops cost
- The honest variable is the automation rate: demos automate 100% of the happy path; production automates **60 to 80%** and escalates the rest

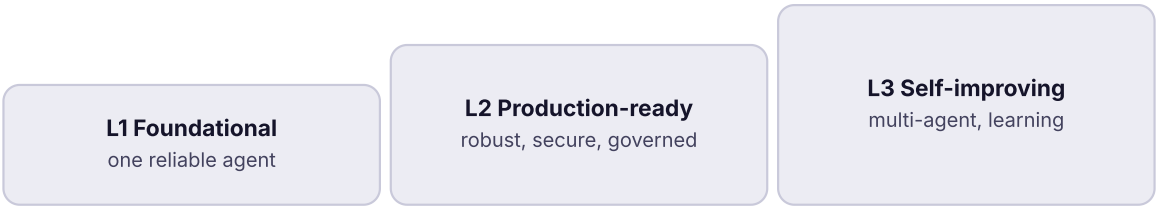
SAY IT

"You can't let one agent both create and judge its own work, so I'd split the generator from the scorer and use R^5 to keep the learning loop safe."

The Roadmap & Frameworks

1. The implementation roadmap

✎ DIAGRAM



Autonomy is earned by evidence, one level at a time.

Implement patterns in this order.

- **Level 1, Foundational**: get one agent working reliably (baseline, memory, sensing with RAG, structured reasoning). Prove value on one well-defined task
- **Level 2, Production-ready**: add fault tolerance (watchdog, retry, fallback, checkpointing), compliance monitoring, authn/authz, human escalation, observability. This is the demo-to-production gap
- **Level 3, Self-improving**: multi-agent coordination, the flywheel with R^5, custom evals, learning patterns

2. The strategic reflection guide

1. Where are you today? (current maturity)
2. What is your **minimum viable agent**? (the smallest reliable Level-1 win)
3. What is your path to production? (the Level-2 patterns you need)
4. What is your North Star? (the Level-3 autonomy you're building towards)

3. Choosing a framework

✎ DIAGRAM

	Google ADK	CrewAI	LangGraph
Core idea	production agents + runtimes	role-playing teams (a crew)	stateful graph (state machine)
Control flow	planner-driven	sequential or hierarchical	edges, cycles, branches
State	managed	implicit	explicit State object
Best for	enterprise, Google ecosystem	standing up a team fast	complex flows, retry loops

The pattern is the architecture; the framework just expresses it.

- **CrewAI**: fastest way to stand up a collaborative team with simple flow
- **LangGraph**: cycles, conditional branching and explicit, inspectable state ('if validation fails, go to the rejection node')
- **ADK**: production runtimes, managed state and callback guardrails, especially in the Google / Vertex ecosystem

SAY IT

"I'd pick CrewAI to stand up a team fast, LangGraph when I need loops and explicit state, and ADK for managed production runtimes, but the pattern is the architecture, not the framework."